

基础作业

1.1 调研：双目相机与结构光相机的深度估计方案

本讲课堂使用的是单目相机图像加几何约束来估计目标深度。实际机器人和视觉系统中，也常使用深度相机直接获得深度信息。

请调研 **双目相机** 和 **结构光相机** 的深度估计方案，并选取两款典型型号进行分析。

要求至少包含以下内容：

- 两款相机的型号名称；
- 每款相机属于哪类深度方案，例如双目、结构光或其他深度方案；
- 每款相机实现深度估计的基本原理；
- 每款相机的典型参数，例如分辨率、深度范围、帧率、视场角、接口类型等；
- 每款相机适合的应用场景；
- 每款相机的优点和局限；
- 简要比较双目相机和结构光相机在深度估计上的差异。

说明：不要求实际运行设备，重点是理解相机的技术方案。

1.2 课堂 Demo 复现：从检测框到机器人坐标系下的位置

按照第十二讲资料包 `robot_vision_coordinate_transform_demo` 的 README，完成课堂实践流程。

需要完成的流程包括：

- 将资料包放入 ROS 工作空间；
- 编译并加载工作空间；
- 启动 Gazebo 静态相机场景；
- 保存一帧 Gazebo 相机图像；
- 读取 Gazebo 中目标真值；
- 运行 YOLO 检测，或在检测不稳定时使用 `config/sample_bbox.yaml`；
- 运行坐标计算流程；
- 得到 `target (u,v)`、`camera_ray`、`P_camera`、`P_robot`；
- 将计算结果与 Gazebo truth 进行对比。

可参考命令：

```
cd ~/ws_gazebo/src/devel/setup.bashroslaunch robot_vision_coordinate_transform_demo camera_demo.launch
```

新开终端保存相机图像：

```
cd ~/ws_gazebo/src/devel/setup.bashroslaunch robot_vision_coordinate_transform_demo save_gazebo_image.py
```

读取 Gazebo 真值：

```
roslaunch robot_vision_coordinate_transform_demo read_gazebo_truth.py
```

进入资料包目录并运行 YOLO / 坐标计算：

```
conda activate robot_yolo
cd ~/ws_gazebo/src/robot_vision_coordinate_transform_demo
python yolo/scripts/detect_image.py --image yolo/images/gazebo_camera.png --no-show
python scripts/run_offline_pipeline.py
```

提供以下验证材料：

- Gazebo 场景截图 1 张；
- 保存相机图像成功的终端截图 1 张；
- YOLO 检测结果截图或使用 sample_bbox.yaml 的说明；
- 坐标计算终端输出截图 1 张，能看到 P_camera、P_robot 和 Gazebo truth；
- 简要说明 P_camera 和 P_robot 分别表示什么。

1.3 修改目标框并分析误差

在课堂 Demo 的基础上，选择 Gazebo 场景中的另一个 box / 目标框，修改程序或配置文件，使其完成深度估计和坐标转换。

说明：YOLO 可能将该目标识别为其他类别，例如 kite 或其他物体类别。本题重点不是类别是否正确，而是 bbox 如何进入坐标计算 pipeline。

要求：

- 说明你选择了哪个目标或哪个 box；
- 给出你使用或修改后的 bbox_xyxy；
- 说明目标被 YOLO 识别成了什么类别，置信度大约是多少；
- 使用该 bbox 计算目标像素点 (u,v)；

- 继续计算 camera_ray、P_camera 和 P_robot;
- 如果有 Gazebo truth 或预期位置, 给出误差; 如果没有真值, 请说明你如何判断结果是否合理;
- 分析误差可能来自哪些原因。

基础作业提交要求

请将基础作业整理为 一份 PDF, 包含:

- 双目相机 / 结构光相机调研结果;
- 两款典型相机型号的原理、参数和应用场景分析;
- 课堂 Demo 复现过程截图;
- P_camera、P_robot 和 Gazebo truth 对比结果;
- 另一个 box / 目标框的坐标计算结果;
- 误差来源分析。

Answers:

1.1 调研：双目相机与结构光相机的深度估计方案

深度估计的核心任务是获取场景中物体到相机的距离信息。目前主流的深度相机技术方案可分为三类：双目立体视觉（被动式）、结构光（主动式）和 ToF 飞行时间法。其中双目视觉和结构光在工业机器人、消费电子、人机交互等领域应用尤为广泛。

双目视觉是一种被动式 3D 成像方式，通过两台相机从不同角度拍摄同一场景，利用左右图像中对应点的视差进行三角测量恢复深度信息。结构光是一种主动式 3D 成像方式，通过红外投影器向物体表面投射已知编码图案（如激光散斑、条纹），再由红外相机采集图案在物体表面的形变，结合三角测量原理计算深度信息。

选择 Stereolabs ZED 2i（双目立体相机）和 Orbbec Astra Pro（结构光相机）对比如下，

项	Stereolabs ZED 2i	Orbbec Astra Pro
深度方案	被动双目视觉+神经网络深度引擎	红外散斑结构光深度方案
基本原理	相机包含左右两个同步成像镜头。系统先进行双目标定和图像校正，然后在左右图像中寻找同一空间点的对应像素，计算视差 d。根据三角测量关系得到深度： $Z = fB / d$ ，其中 f 为焦距，B 为双目基线距离。视差越大，目标越近；视差越小，目标越远。同时利用神经网络深度处理引擎对常规立体匹配深度图进行二次优化。	相机通过红外投影器向场景投射预先设计好的红外散斑或结构光图案，红外摄像头接收物体表面反射后的图案。由于不同距离处图案会产生不同形变或位移，系统可根据已知投影图案、相机标定参数和三角测量关系计算每个像素对应的深度。

	提升了对无纹理表面、反光表面、低光照环境的适应能力。	
典型参数	双目分辨率最高约 2.2K，常用模式包括 1080p@30 fps、720p@60fps；深度范围约 0.2m 到 20 m；视场角 120°（对角线）/ 110°（水平）/ 70°（垂直）；接口为 USB 3.0 Type-C；内置 IMU 等传感器。	USB 2.0 接口下，深度分辨率最高可达 640×480@30fps；深度范围约 0.6m 到 8m，实际稳定工作多用于室内；视场角约 58°（水平）/ 45°（垂直）；接口常见为 USB 2.0。
适用场景	移动机器人导航、室外环境感知、SLAM 建图、无人车/无人机避障、人体检测、三维重建等。	室内机器人、人机交互、手势识别、体感控制、教育实验、近距离三维扫描、服务机器人避障等。
优点	不依赖主动投射光，室外适应性较好；测距范围较远；可同时获得高分辨率彩色图像和深度信息；适合机器人移动感知。	主动投射结构光，不太依赖物体本身纹理；近距离深度图较稳定；成本较低，使用简单；适合室内固定或半固定场景。
局限	对纹理依赖较强，白墙、玻璃、反光表面、弱光环境下匹配困难；需要较强计算资源；近距离精度受基线限制；深度误差会随距离增加而增大。	容易受强阳光和其他红外光干扰；多台结构光相机同时工作时可能互相干扰；有效距离有限；对透明、镜面反光、强吸收红外的材料效果较差。

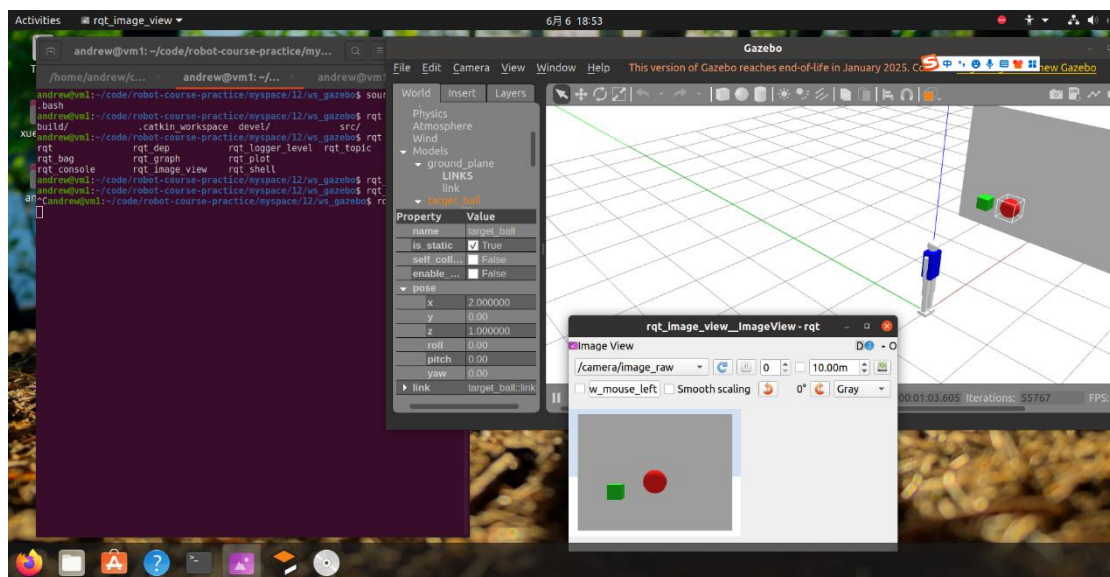
双目相机主要依靠两幅图像之间的视差来估计深度，本质上是“从两个视角看同一物体”进行三角测量。它不一定需要主动光源，因此更适合室外、远距离和移动机器人场景，但对图像纹理、光照和匹配算法要求较高。

结构光相机则主动向场景投射已知图案，再根据图案变形计算深度。它在室内近距离、低纹理物体上的表现通常更稳定，使用也较方便，但容易受到阳光、红外干扰和多相机干扰影响，测距范围通常比双目方案短。

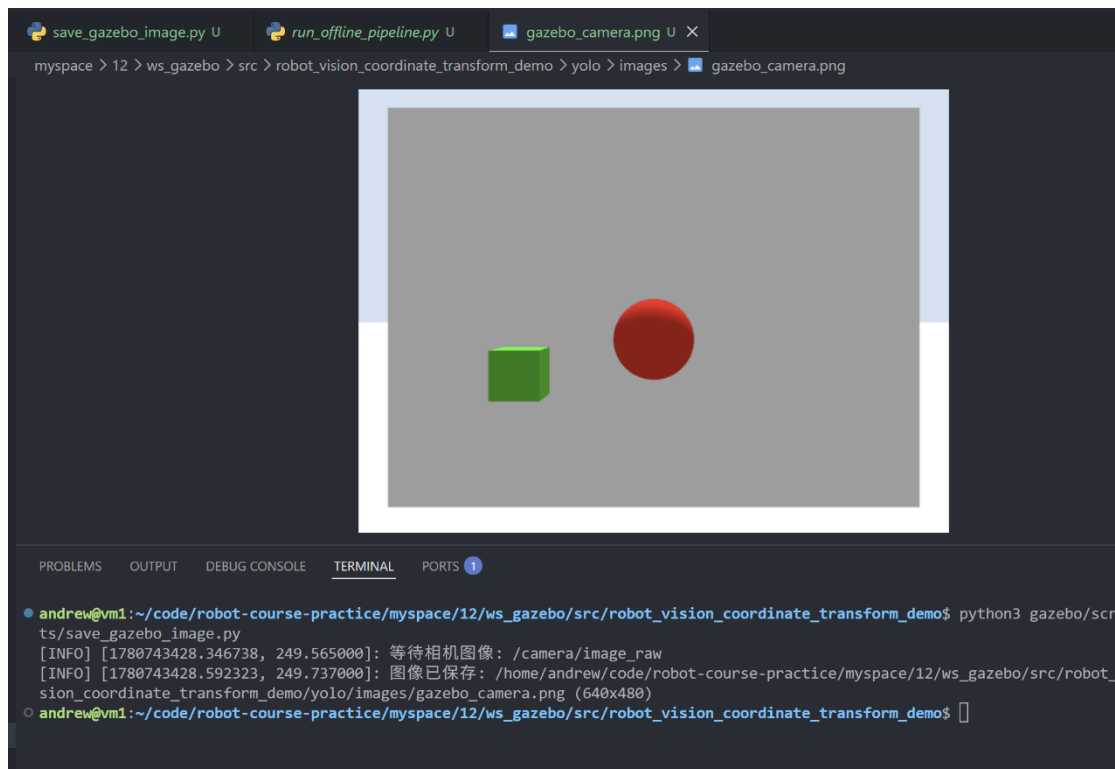
整体上双目方案更适合开放环境、较远距离和机器人自主导航；结构光方案更适合室内、近距离、交互和三维感知任务。

1.2 课堂 Demo 复现：从检测框到机器人坐标系下的位置

(1) Gazebo 场景截图



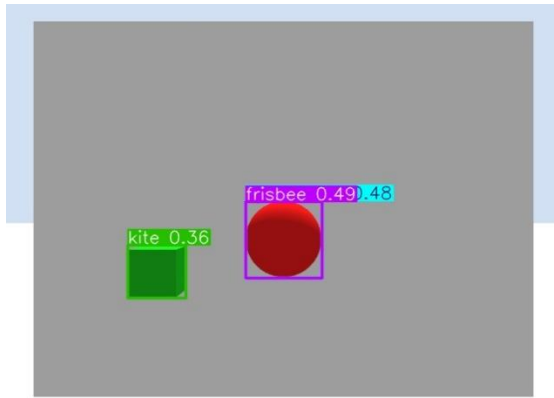
(2) 相机图像保存



(3) YOLO 检测结果截图

YOLO mini anaconda 环境位于 windows 系统上，将相机图像 gazebo_camera.png 下载到 windows 系统进行识别。





(4) 坐标计算终端输出截图

在分析 run_offline_pipeline.py 以及所调用的其他 3 个 py 文件后，将运行以上步骤得到的 target_ball_truth.yaml 下载到 windows 系统上，再执行 run_offline_pipeline.py。

```

Anaconda PowerShell Prompt
(robot_yolo) PS D:\code\xuetangx\robot-course-practice\myspace\12> python .\robot_vision_coordinate_transform_demo\scripts\run_offline_pipeline.py

$ D:\software\miniconda3\envs\robot_yolo\python.exe D:\code\xuetangx\robot-course-practice\myspace\12\robot_vision_coordinate_transform_demo\scripts\compute_target_pixel.py --input D:\code\xuetangx\robot-course-practice\myspace\12\robot_vision_coordinate_transform_demo\yolo\outputs\detections.yaml --fallback D:\code\xuetangx\robot-course-practice\myspace\12\robot_vision_coordinate_transform_demo\config\sample_bbox.yaml
class_name: sports ball
confidence: 0.476
bbbox_xyxy: [276.343, 226.816, 364.628, 315.93]
target_pixel: (u, v) = (320.49, 271.37)
saved to: D:\code\xuetangx\robot-course-practice\myspace\12\robot_vision_coordinate_transform_demo\yolo\outputs\target_pixel.yaml

$ D:\software\miniconda3\envs\robot_yolo\python.exe D:\code\xuetangx\robot-course-practice\myspace\12\robot_vision_coordinate_transform_demo\scripts\compute_camera_ray.py
target_pixel: (u, v) = (320.49, 271.37)
camera_ray: [0.000876, 0.056591, 1.000000]
saved to: D:\code\xuetangx\robot-course-practice\myspace\12\robot_vision_coordinate_transform_demo\yolo\outputs\camera_ray.yaml

$ D:\software\miniconda3\envs\robot_yolo\python.exe D:\code\xuetangx\robot-course-practice\myspace\12\robot_vision_coordinate_transform_demo\scripts\compute_robot_position.py
depth_method: known_size
Z: 1.866 m
P_camera: Xc=0.002, Yc=0.106, Zc=1.866
P_robot: Xr=1.956, Yr=-0.002, Zr=0.999
Gazebo truth: [2.0, 0.0, 1.0]
error norm: 0.044 m
saved to: D:\code\xuetangx\robot-course-practice\myspace\12\robot_vision_coordinate_transform_demo\yolo\outputs\coordinate_transform_result.yaml

=====
离线坐标转换链路汇总
=====
selected class: sports ball
target pixel: u=320.49, v=271.37
depth: method=known_size, Z=1.866 m
P_robot: Xr=1.956, Yr=-0.002, Zr=0.999
truth error: 0.044 m

```

P_camera: sport ball 中心点(u,v)在相机坐标系下的坐标。

P_robot: sport ball 中心点(u,v)在机器人坐标系下的坐标。

1.3 修改目标框并分析误差

(1) 选择 Gazebo 场景中的绿色盒子 (green_reference_box)，对应 YOLO 检测出的 kite 类别，置信度 0.362133。

(2) 修改程序和配置如下，

- read_gazebo_truth.py

```

40 def main():
41
42
43     # model_name = rospy.get_param("~model_name", "target_ball")
44     model_name = rospy.get_param("~model_name", "green_reference_box")
45     default_output = package_dir() / "gazebo_truth" / f"{model_name}_truth.yaml"
46     output = Path(rospy.get_param("~output", str(default_output))).expanduser()

```

运行结果：

```
andrew@vm1:~/code/robot-course-practice/myspace/12/ws_gazebo$ source devel/setup.bash
andrew@vm1:~/code/robot-course-practice/myspace/12/ws_gazebo$ python3 src/robot_vision_coordinate_transform_demo/gazebo/scripts/read_gazebo_truth.py
[INFO] [1780797775.040630, 346.506000]: 等待 /gazebo/model_states ...
[INFO] [1780797775.102220, 346.555000]: Gazebo 真值已保存: /home/andrew/code/robot-course-practice/myspace/12/ws_gazebo/src/robot_vision_coordinate_transform_demo/gazebo_truth/green_reference_box_truth.yaml, position=(2.200, 0.550, 0.850)
```

- compute_target_pixel.py

```
37 def choose_detection(data, prefer_class="sports ball"):
51     if not valid:
52         detail = " ".join(errors)
53         raise ValueError(f"没有可用检测框。{detail}")
54
55     objects = [
56         det for det in valid
57         if str(det.get("class_name", "")).strip().lower() == prefer_class
58     ]
59     if objects:
60         return max(objects, key=detection_confidence)
61
62     return max(valid, key=detection_confidence)

113 def main():
114     args = parse_args()
115     input_path = Path(args.input)
116     fallback_path = Path(args.fallback)
117
118     try:
119         # data, det, source_path = load_detection(input_path, fallback_path)
120         data, det, source_path = load_detection(input_path, fallback_path, "kite")
121     except ValueError as exc:
122         raise SystemExit(str(exc))
123     result = compute_target_pixel(det)
```

将 1.2 部分 windows 运行 YOLO 检测得到的 detections.yaml 结果复制到虚机 Ubuntu 上，再运行 compute_target_pixel.py，运行结果：

```
andrew@vm1:~/code/robot-course-practice/myspace/12/ws_gazebo$ python3 src/robot_vision_coordinate_transform_demo/scripts/compute_target_pixel.py
class_name: kite
confidence: 0.362
bbox_xyxy: [140.557, 277.555, 207.336, 338.852]
target_pixel: (u, v) = (173.95, 308.20)
saved to: /home/andrew/code/robot-course-practice/myspace/12/ws_gazebo/src/robot_vision_coordinate_transform_demo/yolo/output/target_pixel.yaml
andrew@vm1:~/code/robot-course-practice/myspace/12/ws_gazebo$
```

target_pixel.yaml

```
class_name: kite
confidence: 0.362133
bbox_xyxy:
- 140.557
- 277.555
- 207.336
- 338.852
target_pixel:
  u: 173.94650000000001
  v: 308.20349999999996
bbox_size:
  w: 66.77900000000002
  h: 61.29699999999997
image_width: 640
image_height: 480
```

```
source_yaml: /home/andrew/code/robot-course-practice/myspace/12/ws_gazebo/src/robot_vision_coordinate_transform_demo/yolo/outputs/detections.yaml
```

- coordinate_transform_params.yaml

将 real_height_m 更新成 0.22

```
3 # target:
4 #   model_name: "target_ball"
5 #   real_height_m: 0.30
6 #   center_height_robot_m: 1.0
7
8 target:
9   model_name: "green_reference_box"
10  real_height_m: 0.22
```

运行如下命令,

```
andrew@vm1:~/code/robot-course-practice/myspace/12/ws_gazebo$ python3 src/robot_vision_coordinate_transform_demo/scripts/compare_robot_position.py --truth `pwd`/src/robot_vision_coordinate_transform_demo/gazebo_truth/green_reference_box_truth.yaml
depth_method: known_size
Z: 1.990 m
P_camera: Xc=-0.524, Yc=0.245, Zc=1.990
P_robot:  Xr=2.080, Yr=0.524, Zr=0.860
Gazebo truth: [2.2, 0.55, 0.85]
error norm: 0.123 m
saved to: /home/andrew/code/robot-course-practice/myspace/12/ws_gazebo/src/robot_vision_coordinate_transform_demo/yolo/outputs/coordinate_transform_result.yaml
andrew@vm1:~/code/robot-course-practice/myspace/12/ws_gazebo$
```

coordinate_transform_result.yaml

```
depth_method: known_size
depth_Z_m: 1.989715646769011
frames:
  P_camera: 'camera_link optical convention: X right, Y down, Z forward'
  P_robot: 'robot/base simplified frame: X forward, Y left, Z up'
  gazebo_truth: gazebo_world
P_camera:
  Xc: -0.5241980847349791
  Yc: 0.24478799941269555
  Zc: 1.989715646769011
P_robot:
  Xr: 2.079715646769011
  Yr: 0.5241980847349791
  Zr: 0.8602120005873044
comparison:
  gazebo_truth:
    - 2.2
    - 0.55
    - 0.85
  gazebo_truth_frame: gazebo_world
  error_xyz:
    - -0.12028435323098918
    - -0.025801915265020958
    - 0.010212000587304426
  error_norm: 0.12344370951788433
```

(3) 误差分析

类别不匹配: YOLO 将绿色方块识别为 kite 且检测置信度也较低, 表明检测数据本

身存在较大误差。

目标尺寸误差：known_size 方法把框高和真实高度绑定。若使用红球的 0.30m，估计深度会从 1.9897m 增大到 2.713m，误差会显著变大。